

Bản trình chiếu: Đơn giản hóa quan hệ dữ liệu

He Lin

2005

Nguồn gốc: Đơn giản hóa quan hệ dữ liệu: bản trình chiếu

IOI China candidate team papers / 2005 / He Lin / He Lin.ppt

1 Mạch chính của slide

Bản trình chiếu đi cùng bài viết *Đơn giản hóa quan hệ dữ liệu*. Phần chữ còn trích xuất được từ PowerPoint cũ khá ít, nhưng có các mốc

$$\left\lceil \frac{n}{2} \right\rceil, \quad \sum_i \left\lceil \frac{n_i}{2} \right\rceil$$

và từ khóa boat. Những mốc này khớp với ví dụ đi thuyền trong bài viết. Bản ghi chú dưới đây phục hồi mạch slide theo bài viết đã dịch: quan hệ dữ liệu có thể quá rối; nếu biết bỏ phần không cần thiết, ta có thể đổi đồ thị thành cây, rồi đổi cây thành dãy.

Quan hệ giữa dữ liệu quan trọng không kém dữ liệu. Thành phố và đường tạo ra quan hệ đồ thị; cấp trên—cấp dưới tạo ra quan hệ cây; điểm số sắp hạng tạo ra quan hệ tuyến tính. Đề thi thường yêu cầu khai thác thông tin đầu vào, nhưng khai thác mọi quan hệ một cách trực tiếp đôi khi chỉ làm bài toán phức tạp hơn. Mục tiêu của slide là đơn giản hóa vừa đủ: bỏ thông tin thừa nhưng vẫn giữ phần quyết định đáp án.

2 Từ đồ thị sang cây: bài toán đi thuyền

Ví dụ đầu tiên là bài toán thuê thuyền. Có n học sinh, mỗi thuyền chở tối đa hai người. Hai học sinh có thể ngồi chung nếu cùng họ hoặc cùng tên. Cần đưa tất cả lên thuyền với số thuyền ít nhất.

Cách dựng đầy đủ là một đồ thị: mỗi học sinh là một đỉnh; cạnh đỏ nối hai người cùng họ, cạnh xanh nối hai người cùng tên. Muốn ghép từng cặp học sinh thành ít thuyền nhất chính là tìm ghép cực đại. Nhưng đồ thị không nhất thiết hai phía, nên lời giải tổng quát cần thuật toán blossom, vượt quá mục tiêu của slide.

Tác giả quan sát rằng ta không cần giữ mọi cạnh. Với một thành phần liên thông, có thể dựng một cây nhị phân có tính chất: con trái cùng họ với cha, con phải cùng tên với cha. Khi thêm một đỉnh mới vào cây, chỉ cần tìm một đỉnh đã có liên hệ cùng họ hoặc cùng tên, rồi đi theo nhánh tương ứng đến vị trí trống. Nhờ tính liên thông, việc này luôn thực hiện được.

Sau khi có cây, xử lý từ lá lên. Nếu lá P là con duy nhất của cha F , ghép P với F và xóa cả hai. Nếu F có hai con, ta ghép F với một con, còn con kia được nối lại lên phía trên theo quan hệ họ hoặc tên vẫn hợp lệ. Cây luôn giữ được tính chất biểu diễn quan hệ đủ dùng.

Với một thành phần có n_i học sinh, số thuyền cần dùng là

$$\left\lceil \frac{n_i}{2} \right\rceil.$$

Nếu đồ thị ban đầu có nhiều thành phần liên thông, xử lý độc lập và cộng lại:

$$\sum_i \left\lfloor \frac{n_i}{2} \right\rfloor.$$

Đây chính là công thức còn đọc được trong slide. Điểm quan trọng là lời giải không tìm ghép trên toàn đồ thị; nó chỉ giữ một cây đủ chứng minh rằng các học sinh trong cùng thành phần có thể ghép tối ưu.

3 Cây DFS thay cho đồ thị: nhận diện xương rồng

Ví dụ thứ hai là kiểm tra đồ thị có hướng xương rồng. Đồ thị cần liên thông mạnh và mỗi cạnh phải thuộc đúng một chu trình. Cách mô phỏng trực tiếp, liên tục tìm chu trình rồi co lại, rất khó cài vì quan hệ giữa chu trình, đỉnh và cạnh thay đổi sau mỗi lần co.

Slide chuyển quan hệ đồ thị về cây DFS. Nếu đồ thị là xương rồng, cây DFS có những dấu hiệu mạnh:

- không có cạnh ngang giữa hai nhánh DFS;
- với mọi cạnh cây (v, u) , phải có $\text{Low}(u) \leq \text{DFS}(v)$, nếu không sẽ có cầu;
- với mỗi đỉnh v , số nhánh con quay lên tổ tiên thật sự cộng với số cạnh ngược đi ra từ chính v phải nhỏ hơn 2.

Ba điều kiện này kiểm tra được trong một lần DFS. Chúng thay một định nghĩa toàn cục về chu trình bằng quan hệ tổ tiên–hậu duệ và cạnh ngược trong cây. Độ phức tạp là $\mathcal{O}(n + m)$. Đây là ví dụ tốt của việc đơn giản hóa quan hệ: cây DFS không chứa toàn bộ cấu trúc của đồ thị, nhưng chứa đúng phần cần cho kiểm tra.

4 Từ cây sang dãy: thống kê hậu duệ

Phần tiếp theo xử lý quan hệ cây. Với mỗi đỉnh v trong cây đánh số $1, \dots, n$, cần tính số hậu duệ của v có nhãn nhỏ hơn v . Làm trực tiếp trên từng cây con sẽ tốn $\mathcal{O}(n^2)$.

Ta thực hiện DFS và ghi dãy thứ tự thăm. Sau đó đảo thứ tự con của mọi đỉnh và DFS lại, thu được dãy DFS ngược. Hai dãy này biến quan hệ hậu duệ thành giao của hai vùng trong hai trật tự tuyến tính. Vì vậy số hậu duệ nhỏ hơn v có thể tính bằng bao hàm–loại trừ từ ba lượng: số phần tử nhỏ hơn v phía sau v trong dãy DFS, số phần tử nhỏ hơn v phía sau v trong dãy DFS ngược, và số tổ tiên trực tiếp nhỏ hơn v .

Các truy vấn “có bao nhiêu số nhỏ hơn giá trị hiện tại trong một đoạn phía sau” là dạng tuyến tính quen thuộc, giải bằng cây Fenwick hoặc cây đoạn. Do đó toàn bộ bài toán trở thành $\mathcal{O}(n \log n)$. Slide muốn nhấn mạnh rằng dãy không thay thế cây một cách đầy đủ; nó chỉ biến phần quan hệ cần đếm thành dạng có thể dùng cấu trúc dữ liệu chuẩn.

5 Euler tour cho LCA

Một ví dụ kinh điển khác là tổ tiên chung gần nhất. Ta duyệt Euler: mỗi lần vào một cây con và quay lại gốc, ghi lại đỉnh hiện tại. Với mỗi lần xuất hiện, ghi độ sâu. Nếu chọn một vị trí xuất hiện của p và một vị trí xuất hiện của q , thì đỉnh có độ sâu nhỏ nhất trong đoạn giữa hai vị trí ấy chính là LCA của p và q .

Như vậy truy vấn trên cây chuyển thành RMQ trên dãy độ sâu. Có thể dùng cây đoạn để trả lời $\mathcal{O}(\log n)$ mỗi truy vấn, hoặc tiền xử lý RMQ tĩnh để trả lời $\mathcal{O}(1)$. Đây là một ví dụ rất sạch của việc đổi quan hệ: quan hệ tổ tiên trong cây được biểu diễn bằng vị trí và cực tiểu trong một dãy.

6 Kết luận

Đơn giản hóa không phải là vứt bỏ thông tin tùy tiện. Nó là chọn một hình thức biểu diễn yếu hơn nhưng vẫn đủ để quyết định đáp án. Trong bài thuyền, đồ thị được thay bằng cây nhị phân trong từng thành phần. Trong bài xương rồng, đồ thị được nhìn qua cây DFS và các cạnh ngược. Trong các bài cây, quan hệ hậu duệ và LCA được chuyển thành quan hệ trên dây.

Khi gặp quan hệ dữ liệu phức tạp, nên hỏi: phần nào của quan hệ thật sự ảnh hưởng tới đáp án, phần nào chỉ gây nhiễu, và có cấu trúc đơn giản hơn nào giữ được phần cốt lõi đó không. Đó là tư tưởng chính của bản trình chiếu.